NF2 · DOCUMENTACIÓN Y DISEÑO TÉCNICO

AthletaGuard

Sistema Inteligente de Analítica Deportiva y Prevención de Sobrecarga

Proyecto de Inteligencia Artificial y Big Data · Curso 2026-2027

Integrante	Rol principal
David	Project Lead & ML Engineer / Data Scientist
Iván	Backend & Data Pipeline Engineer
Arnau	Frontend Specialist & BI / UI Analyst

Índice

Índice 2
1. Resumen ejecutivo 4
2. Descripción funcional del sistema 4
2.1 Problema que resuelve 4
2.2 Flujo general de uso 4
2.3 Casos principales de usuario 4
CU1 · Consulta de carga por el deportista 4
CU2 · Predicción de sobrecarga 4
CU3 · Supervisión por el entrenador 4
CU4 · Análisis histórico 4
CU5 · Integración técnica 5
2.4 Capacidades y límites 5
3. Alcance definitivo (IN / OUT) 5
3.1 IN · Funcionalidades mínimas obligatorias 5
3.2 IN · Funcionalidades opcionales 5
3.3 OUT · Elementos descartados 5
4. Arquitectura del sistema 5
4.1 Data Layer 6
4.2 Feature Engineering 6
4.3 ML Engineering Layer 6
4.4 API Layer 6
4.5 Persistencia 6
4.6 Frontend Layer 7
5. Tecnologías seleccionadas 7
6. Modelo de datos y diccionario de variables 8
6.1 Entidades, claves y relaciones 8
6.2 Diccionario inicial de variables 8
7. Datos: fuentes, calidad y riesgos 9
7.1 Fuentes candidatas 9
7.2 Calidad esperada y controles 9
7.3 Riesgos específicos de los datos 9
8. Ética, legalidad y accesibilidad 10
8.1 Privacidad y PII 10
8.2 Sesgos y uso responsable 10
8.3 Accesibilidad prevista 10
9. Backlog inicial 10
10. Plan de trabajo y reparto de responsabilidades 10
10.1 Cronograma de ejecución 10
10.2 Dependencias e hitos 11
10.3 Reparto de responsabilidades 11
11. Estrategia de desarrollo y testing 11
11.1 Flujo Git e integración 11
11.2 Validación funcional y testing 11
12. Métricas de evaluación del modelo 12
13. Riesgos técnicos actualizados y Plan B 12
13.1 Orden de actuación si el modelo no alcanza el objetivo 13
14. Repositorio del proyecto y README 13
14.1 Contenido mínimo del README 13
15. Conclusión 14

Criterio de coherencia: este NF2 desarrolla y concreta las decisiones del NF1 sin alterar el objetivo central del proyecto: análisis de series temporales deportivas para estimar fatiga y riesgo de sobrecarga, con un modelo propio y una arquitectura Data → ML → API → Dashboard.

1. Resumen ejecutivo

AthletaGuard es un sistema de Inteligencia Artificial y Big Data orientado al análisis de series temporales deportivas para anticipar situaciones de fatiga y riesgo de sobrecarga. El proyecto parte de métricas de entrenamiento como frecuencia cardíaca, variabilidad de la frecuencia cardíaca (HRV), volumen, percepción subjetiva del esfuerzo (RPE), potencia y descanso. La solución combina un pipeline de ingestión y preprocesamiento, ingeniería de características temporales, entrenamiento de modelos propios de Machine Learning y, de forma opcional, Deep Learning, una API REST de inferencia y un dashboard web de monitorización. El valor principal consiste en transformar datos deportivos dispersos en indicadores comprensibles que ayuden a deportistas y entrenadores a revisar la carga y detectar patrones potencialmente problemáticos. AthletaGuard no realizará diagnósticos médicos ni sustituirá el criterio profesional: sus predicciones serán orientativas y estarán destinadas al apoyo a la decisión.

2. Descripción funcional del sistema

2.1 Problema que resuelve

Los deportistas y entrenadores disponen cada vez de más información procedente de pulsómetros, GPS, relojes deportivos y registros de entrenamiento. Sin embargo, disponer de datos no implica interpretarlos correctamente. La fatiga puede acumularse a lo largo de varios días o semanas y no siempre se detecta observando una única sesión. Cuando el análisis se basa únicamente en reglas fijas o intuición resulta más difícil combinar volumen, intensidad, RPE, HRV y descanso para identificar cambios relevantes.

AthletaGuard añade una capa analítica sobre estos registros. El sistema analizará la evolución temporal, generará características derivadas y aplicará un modelo predictivo para clasificar el nivel de riesgo de sobrecarga. El objetivo es ofrecer una señal temprana que permita revisar la planificación, no decidir automáticamente qué debe hacer el deportista.

2.2 Flujo general de uso

Paso	Operación	Resultado
1	Ingesta	Carga de datasets deportivos o ficheros estructurados.
2	Validación y limpieza	Detección de nulos, duplicados, formatos incoherentes y valores fuera de rango.
3	Feature Engineering	Cálculo de ACWR, medias móviles, variación de HRV, carga reciente y descanso.
4	Entrenamiento / inferencia	Entrenamiento de modelos o aplicación del modelo serializado a nuevos datos.
5	Persistencia	Almacenamiento de sesiones, métricas derivadas y predicciones.
6	Visualización	Dashboard con evolución, nivel de riesgo e históricos.

2.3 Casos principales de usuario

CU1 · Consulta de carga por el deportista

El deportista accede a su histórico y revisa volumen, RPE, frecuencia cardíaca, carga acumulada y tendencias de las últimas semanas.

CU2 · Predicción de sobrecarga

El sistema procesa las variables recientes y devuelve un nivel de riesgo bajo, moderado o alto. La salida incluye fecha de la predicción y aviso de carácter no médico.

CU3 · Supervisión por el entrenador

El entrenador consulta varios deportistas y prioriza la revisión de aquellos con mayor aumento de carga, deterioro de HRV o riesgo estimado elevado.

CU4 · Análisis histórico

El usuario compara periodos anteriores para observar si existen patrones repetidos de fatiga, aumentos de carga o recuperación.

CU5 · Integración técnica

Un cliente del sistema solicita una predicción mediante la API REST y recibe una respuesta JSON con el nivel de riesgo y metadatos básicos del modelo.

2.4 Capacidades y límites

El sistema SÍ podrá	El sistema NO podrá
Importar y procesar datos deportivos.	Realizar diagnósticos médicos.
Generar variables temporales y de carga.	Prescribir tratamientos o rehabilitación.
Entrenar y comparar modelos propios.	Garantizar que una lesión ocurrirá o no.
Emitir alertas orientativas de riesgo.	Modificar automáticamente un plan sin supervisión.
Exponer inferencias mediante API.	Conectarse obligatoriamente en producción a Garmin/Strava.
Mostrar históricos y tendencias.	Utilizar cajas negras comerciales como núcleo del modelo.

3. Alcance definitivo (IN / OUT)

3.1 IN · Funcionalidades mínimas obligatorias

- Ingesta y preprocesamiento de datasets con frecuencia cardíaca, RPE, volumen, frecuencia de sesiones y, cuando exista, HRV o potencia.
- Ingeniería de características con ventanas móviles, medias móviles, ratios de carga aguda/crónica (ACWR), tendencias de HRV y variables de descanso.
- Entrenamiento, ajuste y comparación de modelos Random Forest, XGBoost y/o LightGBM.
- Validación respetando el orden cronológico mediante Time Series Split para minimizar Data Leakage.
- Serialización del modelo seleccionado e integración mediante FastAPI.
- Persistencia en PostgreSQL; SQLite podrá utilizarse en desarrollo local.
- Dashboard web con React para visualizar carga, tendencias, predicciones y alertas.
- Documentación técnica, pruebas básicas y trazabilidad mediante Git/GitHub.

3.2 IN · Funcionalidades opcionales

- LSTM o 1D-CNN si los modelos clásicos no resultan suficientes y existe volumen de datos adecuado.
- Explicabilidad básica de variables mediante importancias o técnicas equivalentes.
- Autenticación y comparación entre varios deportistas.
- Recomendaciones adaptativas de intensidad como extensión, siempre presentadas como apoyo y no prescripción.

3.3 OUT · Elementos descartados

- Integración directa en producción con Garmin o Strava.
- APIs de modelos preentrenados o cajas negras comerciales como motor predictivo principal.
- Diagnóstico médico, tratamiento fisioterapéutico o prescripción clínica.
- Aplicación móvil nativa e infraestructura cloud de pago con arquitectura compleja.

4. Arquitectura del sistema

La arquitectura se organiza en capas para separar responsabilidades y permitir que cada componente pueda desarrollarse, probarse y sustituirse con el menor impacto posible sobre el resto. Esta separación también encaja con el reparto de responsabilidades definido en NF1.

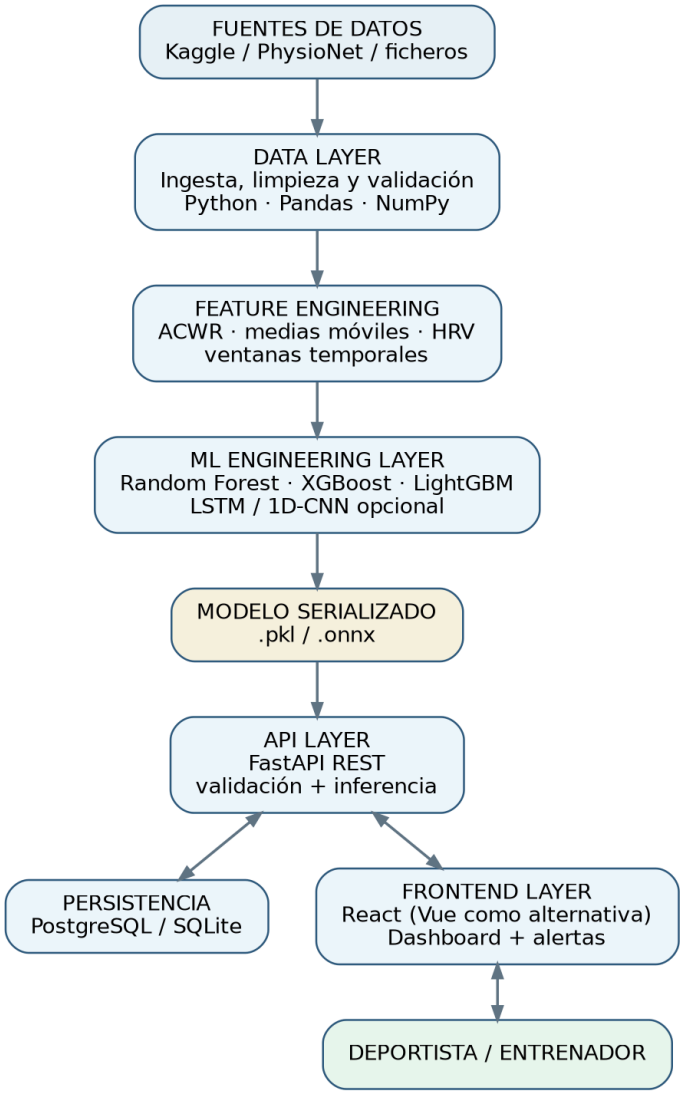


Figura 1. Arquitectura de alto nivel propuesta para AthletaGuard.

4.1 Data Layer

Responsable de extraer, validar, limpiar e imputar los datos. También normaliza unidades y orden temporal. El pipeline debe ser reproducible para garantizar que el mismo tratamiento se aplica durante entrenamiento e inferencia.

4.2 Feature Engineering

Genera variables que resumen la dinámica reciente del entrenamiento: carga de 7 y 28 días, ACWR, medias móviles, variación de HRV, número de sesiones recientes, días de descanso y cambios de volumen entre periodos.

4.3 ML Engineering Layer

Entrena y evalúa los modelos candidatos. La selección final se basará en métricas y estabilidad, no en la complejidad del algoritmo. El resultado se serializa en un formato reutilizable por el backend.

4.4 API Layer

FastAPI recibe datos, valida el esquema de entrada, carga el modelo, ejecuta la inferencia, devuelve JSON y, cuando proceda, almacena la predicción. Endpoints previstos: POST /predict, GET /athletes, GET /athletes/{id}/history y GET /athletes/{id}/risk.

4.5 Persistencia

PostgreSQL será la opción principal para almacenar deportistas, sesiones, métricas derivadas y predicciones. SQLite se reserva como alternativa de desarrollo local para reducir fricción en las primeras pruebas.

4.6 Frontend Layer

React será la opción principal para el dashboard. Mostrará resumen del deportista, evolución temporal, carga, HRV, ACWR, histórico de predicciones y alertas. Vue se mantiene como alternativa si simplifica la implementación del equipo.

Comunicación entre módulos: Frontend ⇄ FastAPI ⇄ PostgreSQL / modelo serializado. El pipeline de entrenamiento opera de forma batch sobre los datasets y genera el artefacto de modelo que consume la API.

5. Tecnologías seleccionadas

Tecnología	Uso	Justificación
Python	Lenguaje principal	Ecosistema consolidado para datos, ML y backend; reduce integración entre capas.
Pandas / NumPy	Data Layer	Adecuados para limpieza, transformación y cálculo vectorizado sobre datos tabulares y temporales.
Scikit-learn	Modelado y evaluación	Aporta pipelines, modelos base, métricas y validación reproducible.
XGBoost / LightGBM	Modelos candidatos	Algoritmos de boosting especialmente útiles para datos tabulares y relaciones no lineales.
TensorFlow / PyTorch	DL opcional	Solo se emplearán si se justifica una LSTM o 1D-CNN por volumen y estructura de datos.
FastAPI	Backend/API	Integración directa con Python, validación de esquemas y documentación automática.
PostgreSQL	Persistencia	Base relacional robusta para entidades con relaciones claras y consultas históricas.
SQLite	Desarrollo local	Alternativa ligera para pruebas iniciales sin servidor.
React	Frontend	Arquitectura por componentes y flexibilidad para dashboards interactivos.
Plotly / Chart.js	Visualización	Gráficos temporales e interactivos adaptados a métricas deportivas.
Jupyter / Colab	Experimentación	EDA, feature engineering y experimentos de entrenamiento reproducibles.
Git / GitHub	Colaboración	Control de versiones, ramas, PR y revisión entre miembros.

Alternativas descartadas o condicionadas: no se fija Deep Learning como requisito porque el NF1 ya lo contempla como alternativa; se utilizará solo si aporta una mejora justificable. Del mismo modo, React se prioriza frente a Vue/Angular para cerrar una opción principal sin impedir un cambio técnico si el equipo detecta una limitación real.

6. Modelo de datos y diccionario de variables

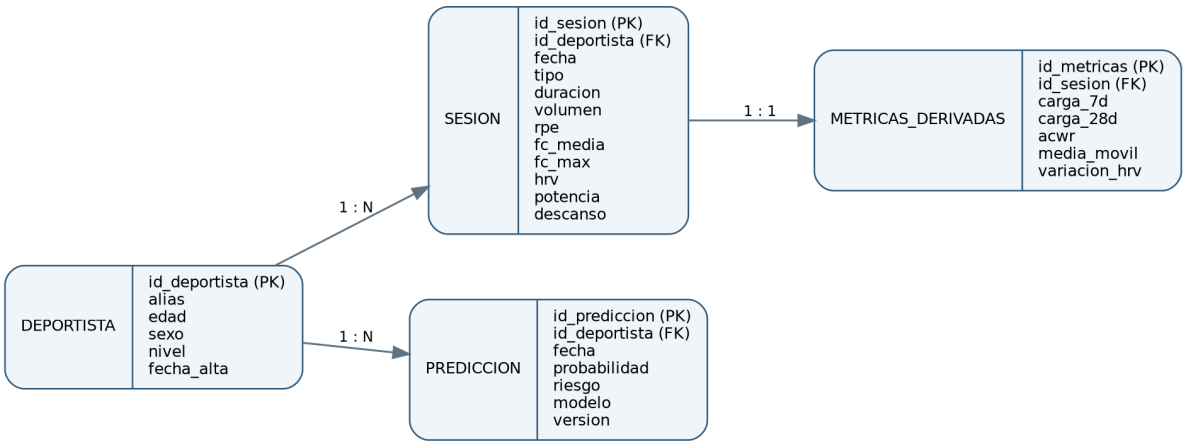


Figura 2. Esquema entidad-relación inicial.

El modelo de datos separa la información original de cada sesión, las métricas calculadas por el pipeline y las predicciones generadas por el modelo. Esta separación facilita recalcular features o sustituir una versión del modelo sin modificar el histórico de entrenamientos.

6.1 Entidades, claves y relaciones

Entidad	Campos principales	Relaciones y finalidad
DEPORTISTA	id_deportista (PK), alias, edad, sexo, nivel, fecha_alta	Entidad principal anonimizada. 1:N con SESION y 1:N con PREDICCION.
SESION	id_sesion (PK), id_deportista (FK), fecha, tipo, duración, volumen, RPE, fc_media, fc_max, HRV, potencia, descanso	Registro base de cada entrenamiento. Pertenece a un deportista.
METRICAS_DERIVADAS	id_metricas (PK), id_sesion (FK), carga_7d, carga_28d, ACWR, media_movil, variacion_hrv	Features calculadas por el pipeline para modelado y visualización.
PREDICCION	id_prediccion (PK), id_deportista (FK), fecha, probabilidad, riesgo, modelo, version	Histórico de inferencias para trazabilidad y comparación de versiones.

6.2 Diccionario inicial de variables

Variable	Tipo	Descripción / uso
fecha	Datetime	Momento de la sesión; base para ordenar y crear ventanas temporales.
duracion	Float	Duración de entrenamiento.
volumen	Float	Medida de volumen disponible en el dataset seleccionado.
RPE	Float	Percepción subjetiva del esfuerzo.
fc_media / fc_max	Float	Frecuencia cardíaca media y máxima de la sesión.
HRV	Float	Variabilidad de la frecuencia cardíaca, cuando esté disponible.
potencia	Float	Potencia registrada, cuando el deporte/dataset la incluya.
descanso	Float	Tiempo o indicador de recuperación entre sesiones.
carga_7d	Float	Carga aguda acumulada en ventana de 7 días.
carga_28d	Float	Carga de referencia de 28 días.

Variable	Tipo	Descripción / uso
ACWR	Float	Ratio entre carga aguda y crónica calculada a partir de ventanas temporales.
variacion_hrv	Float	Cambio de HRV respecto a referencia reciente.
riesgo	Categoría	Variable objetivo o salida final: bajo / moderado / alto según definición del dataset y modelado.

Decisión pendiente controlada: la definición exacta de la etiqueta de sobrecarga dependerá del dataset definitivo. Si la fuente no incluye una etiqueta válida, el equipo reformulará la salida como indicador de carga/fatiga y lo documentará; no se inventará una etiqueta clínica.

7. Datos: fuentes, calidad y riesgos

7.1 Fuentes candidatas

El NF1 identificó como fuentes candidatas datasets públicos de Kaggle relacionados con running, cycling, HRV y training load, así como datos de investigación deportiva disponibles en PhysioNet. En NF2 se mantiene esta decisión, pero el dataset definitivo debe seleccionarse antes de implementar el pipeline de entrenamiento.

Aspecto	Criterio de selección
Licencia	Se verificará la licencia concreta del dataset elegido. Solo se utilizarán datos con condiciones compatibles con un proyecto académico y se citará la fuente exacta.
Tipo	Series temporales tabulares con fecha y métricas de entrenamiento.
Volumen	Preferentemente miles o decenas de miles de registros, suficiente para construir ventanas y separar temporalmente train/validación/test.
Variables	Prioridad a fuentes que incluyan FC, HRV, RPE, volumen/carga, descanso y una etiqueta o proxy justificable.
Actualización	Procesamiento batch; no se requiere streaming ni conexión en tiempo real.
Naturaleza	Datos reales anonimizados cuando la fuente lo permita. Si se utilizan datos simulados o mockeados para pruebas técnicas, se identificarán de forma explícita y no se mezclarán con la evaluación del modelo.

7.2 Calidad esperada y controles

- Comprobación de valores nulos y decisión documentada de imputación o descarte.
- Detección de duplicados y registros repetidos.
- Revisión de tipos, unidades y timestamps.
- Detección de valores fisiológicamente o técnicamente imposibles.
- Comprobación de continuidad temporal y número mínimo de sesiones por deportista.
- Análisis de distribución de la clase objetivo y de posibles sesgos de población.

7.3 Riesgos específicos de los datos

Riesgo	Efecto	Respuesta prevista
Falta de etiqueta fiable	No se puede entrenar una clasificación de sobrecarga defendible.	Seleccionar otra fuente o reformular el objetivo como indicador de carga/fatiga.
Desbalanceo	El modelo puede aprender a predecir siempre la clase mayoritaria.	Class weights/SMOTE solo sobre train y uso de Recall/F1/AUC-PR.
Pocos registros por atleta	Ventanas temporales poco representativas.	Definir mínimo de historial o agrupar análisis según disponibilidad.
Nulos / ruido	Features inestables y pérdida de registros.	Reglas de calidad, imputación documentada y exclusión controlada.

Riesgo	Efecto	Respuesta prevista
Sesgo poblacional	Peor generalización a perfiles poco representados.	Describir composición del dataset y limitar el alcance de las conclusiones.
Data Leakage	Métricas artificialmente altas por uso de información futura.	Orden cronológico, Time Series Split y cálculo de features sin mirar al futuro.

8. Ética, legalidad y accesibilidad

8.1 Privacidad y PII

El prototipo priorizará datasets anonimizados y utilizará identificadores internos. No necesita almacenar DNI, dirección, teléfono o correo personal para cumplir su función. Si en fases posteriores se incorporan datos reales de usuarios, se aplicarán principios de minimización, finalidad, acceso restringido y conservación limitada. El equipo documentará qué campos se tratan y por qué son necesarios.

8.2 Sesgos y uso responsable

La composición del dataset puede condicionar la generalización. Un conjunto centrado en corredores jóvenes o deportistas de cierto nivel no debe presentarse como representativo de toda la población. Por ello se describirán las limitaciones y, cuando el volumen lo permita, se revisará el rendimiento por subgrupos disponibles. Las predicciones serán siempre orientativas y la interfaz mostrará de forma visible que AthletaGuard no constituye un sistema de diagnóstico médico.

8.3 Accesibilidad prevista

- Contraste suficiente entre texto y fondo.
- Tipografía legible y jerarquía clara.
- Gráficos acompañados de valores o etiquetas, evitando depender exclusivamente del color.
- Diseño responsive y navegación sencilla.
- Mensajes de alerta comprensibles, sin lenguaje clínico que induzca a error.

9. Backlog inicial

El backlog inicial contiene diez historias de usuario. Cada una incluye un criterio observable para poder decidir si la funcionalidad está aceptada.

ID	Historia de usuario	Criterios de aceptación
HU01	Como deportista quiero importar mis sesiones para analizar mi carga.	Se cargan registros; se validan campos obligatorios; los errores se informan sin perder los registros válidos.
HU02	Como deportista quiero consultar entrenamientos anteriores.	Lista cronológica y acceso al detalle de una sesión.
HU03	Como entrenador quiero ver la carga semanal de un deportista.	Gráfico por semanas y filtro temporal funcional.
HU04	Como entrenador quiero consultar ACWR y tendencias de carga.	El cálculo se genera desde ventanas definidas y se muestra junto con su fecha.
HU05	Como deportista quiero conocer mi nivel de riesgo estimado.	Se devuelve bajo/moderado/alto, probabilidad cuando aplique y advertencia no médica.
HU06	Como entrenador quiero revisar la evolución de HRV.	Gráfico temporal y comparación con una referencia reciente cuando exista HRV.
HU07	Como sistema quiero detectar datos incorrectos.	Se detectan nulos, duplicados, tipos inválidos y outliers definidos.
HU08	Como desarrollador quiero una API de inferencia.	POST /predict acepta un esquema válido, devuelve JSON y responde adecuadamente ante datos erróneos.
HU09	Como usuario quiero entender el carácter orientativo de la predicción.	Aviso visible en dashboard y respuesta/documentación de la API.
HU10	Como entrenador quiero consultar el histórico de predicciones.	Se muestran fecha, nivel de riesgo y versión del modelo asociada.

10. Plan de trabajo y reparto de responsabilidades

10.1 Cronograma de ejecución

Semana	Objetivo principal	Entregable / hito	Responsable principal
1	Seleccionar dataset definitivo, revisar licencia y variables.	Fuente aprobada + ficha de datos.	David + Iván
2	EDA, limpieza y validación de calidad.	Dataset limpio + informe EDA.	Iván
3	Feature Engineering temporal: ACWR, HRV, medias móviles.	Dataset de modelado reproducible.	David
4	Baseline + Random Forest / XGBoost / LightGBM.	Comparativa inicial de modelos.	David
5	Time Series Split, ajuste y selección.	Modelo candidato + métricas.	David
6	Serialización, esquema PostgreSQL/SQLite y FastAPI.	API de inferencia conectada al modelo.	Iván
7	Dashboard React y visualizaciones.	MVP integrado.	Arnau
8	Testing, corrección, documentación y demo.	Versión candidata a entrega.	Equipo

10.2 Dependencias e hitos

- No puede cerrarse el modelo de datos ni el feature engineering definitivo hasta seleccionar el dataset.
- El backend puede prototiparse con datos mockeados, pero la integración final depende del esquema real de features.
- El dashboard puede construirse con respuestas JSON simuladas antes de que el modelo esté listo.
- El hito crítico es disponer al final de la semana 5 de un modelo serializable con métricas reproducibles.

10.3 Reparto de responsabilidades

Miembro	Responsabilidad principal	Apoyo / revisión cruzada
David	Arquitectura ML, feature engineering, entrenamiento, ajuste y evaluación.	Revisa definición de datos y esquema de inferencia.
Iván	Ingesta, limpieza, persistencia, FastAPI y serialización/integración.	Apoya reproducibilidad del pipeline y pruebas de modelo.
Arnau	React, dashboard, gráficos, UX y presentación de alertas.	Valida requisitos de API y accesibilidad de la interfaz.

El equipo mantendrá dos reuniones semanales de seguimiento, tal y como se estableció en NF1: una revisión corta de estado/bloqueos y una sesión de integración o planificación.

11. Estrategia de desarrollo y testing

11.1 Flujo Git e integración

Se mantiene la regla del NF1: no se hará push directo a main. Las funcionalidades se desarrollarán en ramas y se integrarán mediante Pull Request con revisión de al menos un compañero. Se propone una rama develop para integración previa, sin que ello impida simplificar el flujo si el equipo detecta que añade complejidad innecesaria.

Rama	Uso
main	Versión estable y demostrable.
develop	Integración de funcionalidades antes de pasar a main.
feature/data-pipeline	Ingesta, limpieza y persistencia.
feature/ml	Feature engineering y modelos.
feature/api	FastAPI e inferencia.

Rama	Uso
feature/frontend	Dashboard React y visualizaciones.

11.2 Validación funcional y testing

Área	Pruebas mínimas
Datos	Tipos, nulos, duplicados, rangos, orden temporal y generación de features.
Machine Learning	Separación temporal, ausencia de leakage, métricas reproducibles y comparación train/validación/test.
API	Petición válida, esquema inválido, recurso inexistente, carga del modelo y formato JSON de salida.
Persistencia	Inserción/consulta de sesiones y predicciones, claves y relaciones.
Frontend	Carga de datos, filtros, gráficos, errores de API, mensajes de alerta y aviso no médico.
Integración	Flujo completo desde entrada de datos hasta visualización de una predicción.

12. Métricas de evaluación del modelo

El NF1 ya identificó el desbalanceo como riesgo. Por ello Accuracy no será la única métrica. Se evaluará especialmente la capacidad de detectar los casos minoritarios sin generar un número excesivo de falsos positivos.

Métrica	Motivo de uso
Recall	Mide qué proporción de casos reales de riesgo detecta el modelo; es importante si omitir una señal relevante resulta costoso.
Precision	Permite controlar cuántas alertas emitidas son realmente correctas.
F1-Score	Equilibra Precision y Recall y resulta útil ante clases desbalanceadas.
AUC-PR	Resume el comportamiento Precision-Recall y es informativa cuando la clase positiva es minoritaria.
Matriz de confusión	Permite visualizar falsos positivos y falsos negativos de forma directa.

Validación temporal: las métricas se calcularán sobre particiones que respeten el orden cronológico. El cálculo de ventanas y normalizaciones también debe evitar utilizar datos posteriores al instante que se pretende predecir.

13. Riesgos técnicos actualizados y Plan B

Riesgo	Prob.	Impacto	Mitigación / Plan B
Dataset sin etiqueta de sobrecarga fiable	Alta	Muy alto	Cambiar de fuente o reformular el objetivo como fatiga/carga sin afirmar una predicción clínica.
Desbalanceo de clases	Alta	Alto	Pesos de clase, SMOTE aplicado solo al entrenamiento, ajuste de umbral y métricas Recall/F1/AUC-PR.
Data Leakage temporal	Media	Muy alto	Time Series Split, orden cronológico y features calculadas únicamente con pasado.
Bajo rendimiento del modelo	Media	Alto	Baseline, ajuste de hiperparámetros, XGBoost/LightGBM y, si se justifica, LSTM/1D-CNN.
Overfitting por atleta o por pocas muestras	Media	Alto	Reducir complejidad, limitar features, validación temporal y análisis de diferencia train/test.

Riesgo	Prob.	Impacto	Mitigación / Plan B
Datos faltantes / ruido	Alta	Medio	Reglas de calidad, imputación documentada y exclusión controlada de registros inviables.
Incompatibilidad entre pipeline y API	Media	Medio	Guardar el preprocesamiento junto al modelo o versionar explícitamente features/esquema.
Problemas de integración frontend-backend	Media	Medio	Contrato JSON definido temprano y uso de respuestas mockeadas antes de la integración final.
Exceso de alcance	Media	Alto	Priorizar MVP: datos + modelo + API + dashboard; extras quedan condicionados a tiempo.
Retraso o bloqueo de un miembro	Media	Medio	PR frecuentes, documentación, reuniones y revisión cruzada para evitar conocimiento aislado.
Dependencias/versiones	Baja	Medio	requirements.txt/environment y fijación de versiones antes de la demo final.

13.1 Orden de actuación si el modelo no alcanza el objetivo

El Plan B no consistirá en saltar directamente a una red neuronal. Se seguirá un orden progresivo: (1) revisar calidad y etiqueta, (2) comprobar leakage y partición temporal, (3) revisar features, (4) ajustar pesos/umbral e hiperparámetros, (5) comparar modelos de boosting y (6) valorar LSTM/1D-CNN únicamente si los datos y el tiempo lo justifican.

14. Repositorio del proyecto y README

El repositorio del equipo ya existe y continuará siendo el punto común de trabajo. Se reorganizará para que la documentación, los datos, los experimentos y el código queden separados de forma comprensible.

```
AthletaGuard/
├── README.md
├── .gitignore
├── requirements.txt
├── docs/
│   ├── NF1_Presentacion_alineacion_viabilidad.pdf
│   ├── NF2_Documentacion_y_Diseno_Tecnico.pdf
│   ├── arquitectura.png
│   ├── modelo_datos.png
│   └── experiments/
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
│   ├── 01_EDA.ipynb
│   ├── 02_feature_engineering.ipynb
│   └── 03_model_training.ipynb
├── src/
│   ├── data/
│   ├── features/
│   ├── models/
│   ├── api/
│   ├── backend/
│   └── frontend/
├── tests/
└── environment/
```

14.1 Contenido mínimo del README

- Descripción y objetivo de AthletaGuard.
- Integrantes y responsabilidades.
- Arquitectura resumida Data → Feature Engineering → ML → API/Persistencia → Dashboard.
- Tecnologías utilizadas.

- Estructura del repositorio.
- Instrucciones previstas de ejecución, aunque en NF2 todavía no exista código funcional completo.
- Estado actual y limitaciones principales del proyecto.

Repositorio de referencia del equipo: https://github.com/davidillo99/AthletaGuard (según el acuerdo de trabajo entregado en NF1).

15. Conclusión

AthletaGuard evoluciona en este NF2 desde una propuesta viable hacia un diseño técnico definido y ejecutable. La solución mantiene el objetivo del NF1: procesar series temporales deportivas y utilizar modelos propios de Machine Learning para estimar fatiga y riesgo de sobrecarga, integrando los resultados en una API y un dashboard de monitorización.

La arquitectura separa datos, ingeniería de características, modelado, persistencia, API y frontend, lo que facilita tanto el reparto del equipo como las pruebas independientes de cada módulo. Los principales riesgos -desbalanceo, ausencia de una etiqueta fiable y Data Leakage- se incorporan explícitamente a la planificación para evitar que la calidad del resultado dependa únicamente de la elección del algoritmo.

El MVP se considerará completo cuando sea posible ejecutar el ciclo datos → procesamiento → modelo → API → visualización de forma reproducible. Las funcionalidades opcionales, como LSTM/1D-CNN o recomendaciones avanzadas, se incorporarán únicamente si el núcleo obligatorio está estable. De este modo, el proyecto conserva ambición técnica sin perder realismo ni superar el alcance académico definido.

Documento de planificación técnica · NF2